WATER TANK LEVEL MONITORING AND USAGE SYSTEM

A PROJECT REPORT

Submitted by

GUGANRAJ R-8115U25CS050

in partial fulfillment of requirements for the award of the course

CGB1121 – PYTHON PROGRAMMING

in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Mrs. R. INDRADEVI

AP/CSE

K.RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

SAMAYAPURAM, TRICHY 621112

ANNA UNIVERSITY

CHENNAI 600025

JUNE 2026

WATER TANK LEVEL MONITORING AND USAGE SYSTEM

A PROJECT REPORT

Submitted by

GUGANRAJ R-8115U25CS050

in partial fulfillment of requirements for the award of the course

CGB1121 – PYTHON PROGRAMMING

in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Mrs. R. INDRADEVI

AP/CSE

K.RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Under

ANNA UNIVERSITY, CHENNAI 600025

JUNE 2026

K.RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

SAMAYAPURAM – 621 112

BONAFIDE CERTIFICATE

Certified that this project report titled **"WATER TANK LEVEL MONITORING AND USAGE SYSTEM**" is the bonafide work of **(GUGANRAJ R-8115U25CS050)**, who carried out the project work under my supervision.

SIGNATURE

R. INDRADEVI, M.E.,
SUPERVISOR

ASSISTANT PROFESSOR
DEPARTMENT OF CSE
K.RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
SAMAYAPURAM–621112.

SIGNATURE

Dr. K. CHELLAMUTHU, M.E., Ph.D.,
HEAD OF THE DEPARTMENT

ASSISTANT PROFESSOR
DEPARTMENT OF SCIENCE AND HUMANITIES
K.RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
SAMAYAPURAM–621112.

Submitted for the end semester examination held on …….

SIGNATURE
INTERNAL EXAMINER
NAME:

SIGNATURE
EXTERNAL EXAMINER
NAME:

K.RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Under

ANNA UNIVERSITY, CHENNAI

DECLARATION BY THE CANDIDATE

I jointly declare that the project report on **"WATER TANK LEVEL MONITORING AND USAGE SYSTEM"** is the result of original work done by us and best of our knowledge. This project report is submitted on the partial fulfillment of the requirement of the award of the course **CGB1121 – PYTHON PROGRAMMG.**

Signature

GUGANRAJ R

Place: Samayapuram

Date:

ACKNOWLEDGEMENT

I thank the almighty GOD, without whom it would not have been possible for us to complete our project. I wish to address our profound gratitude to **Dr.K.RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering (Autonomous), who encouraged and gave us all help throughout the course.

I extend my hearty gratitude and thanks to our honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.,** K. Ramakrishnan College of Engineering (Autonomous).

I would like to thank **Dr. D. SRINIVASAN, M.E., Ph.D., FIE., MIIW., MISTE., MISAE., C. Engg.,** Principal, who gave the opportunity to frame the project to full satisfaction.

I wish to convey my heartfelt thanks to **Dr.K.CHELLAMUTHU, M.E., Ph.D.,** and Head of the Department, Science and Humanities for giving fullest support to complete this project.

I wish to convey our profound and heartfelt gratitude to our esteemed project guide **Mrs. R. INDRADEVI, M.E.,** Department of **COMPUTER SCIENCE AND ENGINEERING** for her incalculable suggestions, creativity, assistance and patience, which motivated us to carry out this project.

I render our sincere thanks to the Course Coordinator and other staff members for providing valuable information during the course.

I wish to express our special thanks to the officials and Lab Technicians of our departments who rendered their help during the period ofthe work progress.

INSTITUTE VISION AND MISSION

VISION OF THE INSTITUTE:

To achieve a prominent position among the top technical institutions.

MISSION OF THE INSTIITUTE:

M1: To bestow standard technical education par excellence through state of the art infrastructure, competent faculty and high ethical standards.

M2: To nurture research and entrepreneurial skills among students in cutting edge technologies.

M3: To provide education for developing high-quality professionals to transform the society.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources,

and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO8 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO9 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO10 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO11 Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

This project focuses on the development of a Water Tank Level Monitoring and Usage System using Python. The main aim of this system is to monitor the water level in a tank and control the motor automatically to ensure efficient water management. Water is an essential resource, and improper usage can lead to wastage and shortage. This project helps in solving such problems by providing an automated solution.The system works by continuously checking the water level in the tank. In this project, the water level is simulated using Python variables and conditions. Based on the level of water, the system makes decisions using conditional statements. When the water level is low, the motor is automatically turned ON to fill the tank. When the tank reaches its maximum level, the motor is turned OFF to prevent overflow. This automation reduces the need for human intervention.In addition to monitoring, the system also considers water usage. It can be extended to track how much water is used over time, which helps users understand their consumption patterns. The output is displayed on the screen, showing the current water level and motor status, making it easy for users to monitor the system.This project is simple, cost-effective, and easy to implement. It can be used in homes, agricultural fields, and industries for efficient water management. The system helps in saving water, reducing electricity usage, and preventing water overflow. Overall, this project demonstrates how Python programming can be used to create practical and useful real-world-applications.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1	**INTRODUCTION**	
	1.1 Introduction	12
	1.2 Purpose and importance	12
	1.3 Objective	13
	1.4 Project summarization	13
2	**PROJECT METHODOLGY**	
	2.1 Introduction to System architecture	16
	2.2 Detailed System Architecture Diagram	17
3	**PYTHON PREFERENCE**	
	3.1 Explanation why tuple was chosen	19
	3.2 Key Features of Python	20
4	**DATA STRUCTURE USED**	
	4.1 String	26
	4.2 Tuple	26
	4.3 List	26
5	**MODULES**	
	5.1 Modules	27
	5.2 Modules Description	27

6 CONCLUSION AND FUTURE SCOPE

6.1 Conclusion 31

6.2 Future Scope 32

APPENDICES

Appendix A-Source Code 34

Appendix B-Screenshots 40

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
2.1	Architecture Diagram	18

CHAPRTER 1 – INTRODUCTION

1.1 Introduction

Building a The Water Tank Level Monitoring and Usage System is designed to provide an efficient solution for managing water in tanks. In many homes, farms, and industries, water overflow and shortage are common problems due to the lack of proper monitoring. Manual checking of water levels is time-consuming and may lead to wastage of water and electricity.This project aims to automate the process of monitoring water levels and controlling the motor using Python programming. The system continuously checks the water level in the tank and makes decisions based on predefined conditions. When the water level falls below a certain limit, the motor is automatically turned ON to fill the tank. Similarly, when the tank reaches its maximum capacity, the motor is turned OFF to prevent overflow.

1.2 Purpose and Importance

The primary purpose of this project is to design and implement an automated water tank level monitoring and usage system using Python. The system aims to continuously observe the water level in a tank and control the motor operation without human intervention. By using logical conditions in the program, the system ensures that the motor is switched ON when the water level drops below a minimum threshold and switched OFF when the tank reaches its maximum capacity. This helps in maintaining a balanced water level at all times.

Another important purpose of this project is to analyze water usage patterns. The system keeps track of how much water is consumed and how frequently the tank is filled. This information can help users understand their daily water usage and make better decisions to conserve water.The importance of this project is seen in its ability to prevent water overflow, which can lead to

wastage and damage. It also avoids dry running of the motor, which can cause damage and reduce its lifespan. By automating the process, the system reduces the need for manual checking and saves time and effort.Furthermore, this system contributes to energy efficiency by ensuring that the motor runs only when necessary. This leads to reduced electricity consumption and lower energy costs. The project is also environmentally beneficial as it promotes water conservation and responsible usage.

In addition, the system is simple, cost-effective, and easy to implement, making it suitable for homes, agricultural fields, and small industries. It can also be further enhanced by integrating sensors and IoT technologies for real-time monitoring and remote control. Overall, this project highlights the practical application of programming in solving everyday problems and improving resource management.

1.3 Objective

- To design a system that monitors the water level in a tank continuously.
- To automatically control the motor based on the water level.
- To prevent water overflow and avoid water wastage.
- To reduce manual effort by automating the tank monitoring process.
- To track water usage and maintain usage history.
- To display real-time information about water level and motor status.

1.4 Project Summarization User Interface:

The User Interface of the Water Tank Level Monitoring and Usage System is designed using a simple console-based approach in Python. It provides an easy and interactive way for users to operate the system without requiring technical knowledge..

Database

The system uses Python lists to store data instead of a real database. Water usage and tank filling details are saved in lists as tuples. This data is stored temporarily and is available only while the program is running. It is simple and easy to use but does not store data permanently.

Business Logic

The business logic of this system is based on simple if-else conditions in Python. The program checks the water level and controls the motor automatically. If the water level is low, the motor turns ON. If the tank is full, the motor turns OFF. The system also updates water usage and stores the data in lists

Analytics Module

The Analytics Module plays an important role in the Water Tank Level Monitoring and Usage System by analyzing the data related to water usage and tank filling. This module uses the stored data from the system to provide meaningful insights about water consumption patterns.

In this project, the system maintains two main data structures: usage history and fill history. These are implemented using Python lists, where each entry is stored as a tuple containing the amount of water used or filled along with the corresponding time. This structured storage allows the system to keep track of past activities in an organized manner.The Analytics Module processes this data to calculate the total amount of water used and the total amount of water filled over a period of time.

It can also display the history records to the user, making it easier to review previous usage and understand consumption behavior. By analyzing this information, users can identify patterns such as frequent water usage or unnecessary wastage.This module helps in promoting efficient water management by providing useful insights. Users can make informed decisions to reduce water consumption and improve usage efficiency. Although the current system performs basic analysis, it can be further enhanced by adding features such as graphical representation, daily or monthly reports, and predictive analysis.

Overall, the Analytics Module enhances the functionality of the system by converting stored data into useful information, making the project more practical and effective in real-world applications.

CHAPTER 2 - PROJECT METHODOLOGY

2.1 Introduction to System Architecture

The System Architecture of the Water Tank Level Monitoring and Usage System describes how the different components of the system are organized and how they interact with each other. It provides a clear structure of the system, showing the flow of data from input to processing and output.In this project, the architecture is divided into three main parts: input, processing, and output. The input is provided by the user in the form of water level values or water usage details. Since the system is a simulation, these inputs are entered manually through the console.

2.1.1 High-Level System Architecture

The high-level system architecture for the supply chain management for a grocery store typically consists of several key components:

1. User Interface (UI)
2. Application Logic

2.1.2 Components of the System Architecture

User Interface (UI)

The User Interface (UI) is an important component of the system architecture that allows the user to interact with the Water Tank Level Monitoring and Usage System. In this project, the UI is designed as a simple console-based interface using Python.The UI provides a menu-driven system where users can select different options such as checking the water level, using water, filling the tank, and viewing usage history. The user enters their choice through the keyboard, and the system responds accordingly. It acts as a bridge between the user and the processing unit. The UI collects input from the user and sends it to the system for processing. After processing, it displays the output.

Application Logic

1. Water Level Checking Logic

This logic continuously checks the current water level stored in a variable (like level). The program compares this value with predefined limits such as LOW = 30 and HIGH = 90.

2. Motor Control Logic

- If water level ≤ LOW → motor is turned ON
- If water level ≥ HIGH → motor is turned OFF
- The motor status is stored using a boolean variable (motor_on = True/False).

3. Display Logic

This logic shows output using print():

- Water level
- Motor status
- Alerts like "Tank Full"

Data Management Layer

The Data Management Layer stores and manages data using Python lists and tuples. It keeps water usage and filling records temporarily during program execution.

2.1.3 Detailed System Architecture Diagram

The Detailed System Architecture Diagram represents the complete working flow of the Water Tank Level Monitoring and Usage System. It shows how different modules interact with each other in a continuous loop to monitor and control the water level efficiently.

The system starts with the **Sensor Module**, which reads the current water level using the read_sensor() function. This data is passed to the **Monitoring Module**, where the monitor() function analyzes and displays the water level status.

The processed data is then sent to the **Control Module**, where the control(level) function makes decisions to turn the motor ON or OFF based on the water level. This ensures automatic filling and prevents overflow.

Next, the **Display Module** shows the system status using the display() function, including water level and motor condition. At the same time, the **Alert Module** generates warnings using the alert() function if the water level is too low or too high.

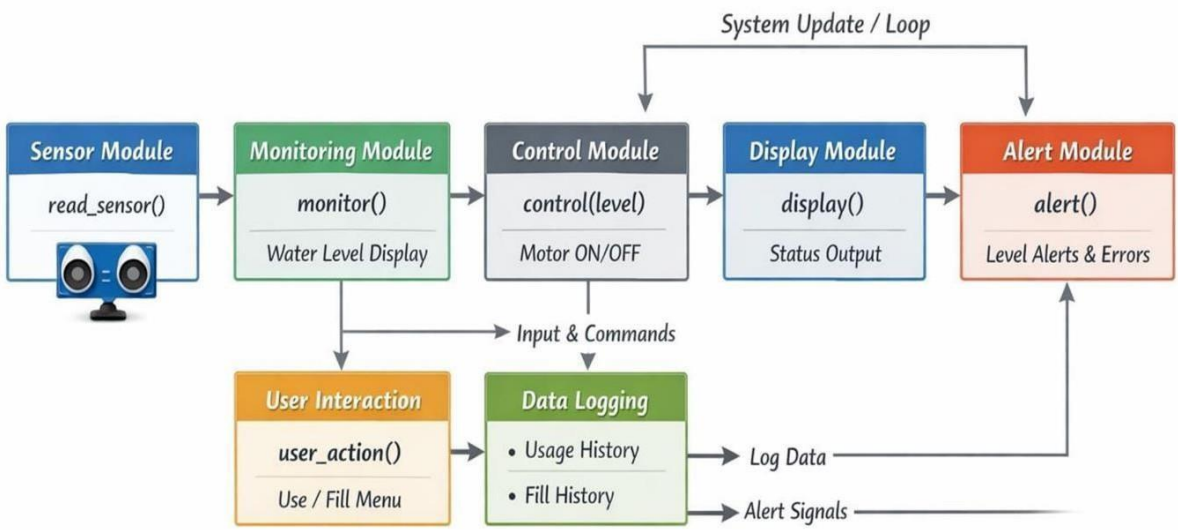


Figure 2.1 : Architecture Diagram (Sample)

CHAPTER 3 - PYTHON PREFERENCE

3.1 Explanation of why a tuple was chosen

A tuple was chosen in this project to store data because it is immutable, meaning its values cannot be changed after creation. This ensures that records such as water usage and filling history remain safe and cannot be accidentally modified.Tuples are also useful for storing related data together, such as (amount, time), in a single structure. They are lightweight, faster than lists, and suitable for fixed data records, making them ideal for maintaining reliable and structured history in the system.

3.2 Comparison with other datatypes

A tuple is compared with other data types like lists, strings, and dictionaries based on its structure, flexibility, and usage. Unlike lists, tuples are immutable, meaning once created, their values cannot be modified, which ensures data integrity and prevents accidental changes in important records such as water usage and filling history. Lists, on the other hand, are mutable and allow adding, removing, or updating elements, making them suitable for storing multiple dynamic records. Compared to strings, which can only store textual data, tuples are more powerful as they can hold different data types together, such as integers and strings in a single structure like (amount, time). When compared to dictionaries, tuples are simpler and consume less memory, whereas dictionaries store data in key-value pairs and are more flexible but slightly complex. Tuples also offer faster access and better performance when dealing with fixed data. Therefore, in this project, tuples are preferred for storing individual records safely and efficiently, while other data types like lists are used for managing collections of such records. Additionally, tuples improve data reliability in the system by ensuring that once a record is stored, it remains unchanged throughout the program execution.

3.3 Features in Python

1. Free and Open Source: Python language is freely available at the official website and you can download it from the given download link below click on the Download Python keyword. Download Python Since it is open-source, this means that source code is also available to the public. So you can download it, use it as well as share it.

2. Easy to code: Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, Javascript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days. It is also a developer-friendly language. 3. Easy to Read

3. Object-Oriented Language: One of the key features of Python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, object encapsulation, etc.

4. GUI Programming Support: Graphical User interfaces can be made using a module such as PyQt5, PyQt4, wxPython, or Tk in Python. PyQt5 is the most popular option for creating graphical apps with Python.

5. High-Level Language: Python is a high-level language. When we write programs in Python, we do not need to remember the system architecture, nor do we need to manage the memory.

6. Large Community Support: Python has gained popularity over the years. Our questions are constantly answered by the enormous StackOverflow community. These websites have already provided answers to many questions about Python, so Python users can consult them as needed.

7. Easy to Debug: Excellent information for mistake tracing. You will be able to quickly identify and correct the majority of your program's issues

once you understand how to interpret Python's error traces. Simply by glancing at the code, you can determine what it is designed to perform.

8. Python is a Portable language: Python language is also a portable language. For example, if we have Python code for Windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

9. Python is an Integrated language: Python is also an Integrated language because we can easily integrate Python with other languages like C, C++, etc.

10. Interpreted Language: Python is an Interpreted Language because Python code is executed line by line at a time. like other languages C, C++, Java, etc. there is no need to compile Python code this makes it easier to debug our code. The source code of Python is converted into an immediate form called bytecode.

11. Large Standard Library : Python has a large standard library that provides a rich set of modules and functions so you do not have to write your own code for every single thing. There are many libraries present in Python such as regular expressions, unit-testing, web browsers, etc.

12. Dynamically Typed Language: Python is a dynamically-typed language. That means the type (for example- int, double, long, etc.) for a variable is decided at run time not in advance because of this feature we don't need to specify the type of variable.

13. Frontend and backend development: With a new project py script, you can run and write Python codes in HTML with the help of some simple tags <py-script>, <py-env>, etc. This will help you do frontend development work in Python like javascript. Backend is the strong forte of Python it's extensively used for this work cause of its frameworks like Django and Flask.

15. Allocating Memory Dynamically: In Python, the variable data type does not need to be specified. The memory is automatically allocated to a variable at runtime when it is given a value.

CHAPTER 4 - DATA STRUCTURE METHODOLOGY

4.1 String

String is a data structure in Python Programming that represents a sequence of characters. It is an immutable data type, meaning that once you have created a string, you cannot change it. Python String are used widely in many different applications, such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text.

4.2 Tuple

Tuples are beneficial in a Python chess game for representing immutable collections of related data, such as a piece's position (row, column), a move (start square, end square), or a specific board configuration. Their immutability ensures that these fundamental data structures remain consistent throughout the game logic, preventing accidental modifications that could lead to errors or inconsistencies in the game state. This inherent safety and the lightweight nature of tuples make them suitable for representing fixed sets of information that are frequently accessed but rarely changed during the execution of the game.

4.3 List

A list is a versatile and dynamic data structure that can store multiple values of various data types, including strings, integers, floats, and other lists. Lists are denoted by square brackets [] and are mutable, meaning they can be modified after creation. You can perform various operations on lists, such as indexing, slicing, appending, inserting, and removing elements. Lists also support various methods like sort(), reverse(), and index(), making them a fundamental and widely used data structure in Python programming.

CHAPTER 5 - MODULES

5.1 Modules Used:

1) Sensor
2) Water Level Monitoring
3) Motor Control
4) Alert
5) Data / Usage Tracking

5.2 Modules Description:

1. **Sensor :** The sensor module reads the current water level from the system. It collects real-time data from the tank and sends this information to the water level monitoring module. It acts as the input device of the system. This helps in accurate detection of water level.

2. **Water Level Monitoring :** The water level monitoring module checks and determines the water level status. It identifies whether the level is low, normal, or full based on sensor data. It processes the input and sends signals to the motor control module. This ensures proper monitoring of water levels.

3. **Motor Control :** The motor control module controls the operation of the water pump. It turns ON the motor when the water level is low and turns it OFF when the tank is full. This automation helps in efficient water management. It reduces manual work.

4. **Display :** The display module shows the system information to the user. It

displays water level, motor status, and system messages. This helps the user to easily monitor the system. It improves user interaction.

4. **Alert :** The alert module provides warning messages to the user. It alerts when the water level is low or when the tank is full. Alerts can be in the form of messages or sounds. This helps in taking quick action

5. **Data / Usage Tracking :** The data or usage tracking module stores and tracks water usage information. It records motor working time and water filling history. This data can be used for future analysis. It helps in better water management

CHAPTER 6 - CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The Water Tank Level Monitoring and Usage System developed using Python is an efficient and practical solution designed to automate the process of monitoring water levels and controlling the motor operation. The main goal of this project is to reduce manual effort, prevent water wastage, and ensure proper utilization of water resources. In real-life scenarios such as homes, agriculture, and industries, water management is a critical issue. This project addresses these problems by introducing an automated system that continuously monitors water levels and takes appropriate actions based on predefined conditions.

The system works by simulating water level inputs and processing them using Python programming logic. When the water level falls below a minimum threshold, the system automatically turns the motor ON to fill the tank. When the water level reaches the maximum limit, the motor is turned OFF to prevent overflow. This simple decision-making mechanism ensures that the tank maintains an optimal water level at all times without requiring constant human supervision. As a result, both water and electricity are saved effectively.

One of the major achievements of this project is automation. The system operates automatically based on conditions defined in the program using if-else statements. This reduces the need for manual monitoring and makes the system more reliable and efficient. Automation also helps in avoiding human errors such as forgetting to turn off the motor, which can lead to water overflow and wastage.

Another important feature of this project is water usage tracking. The system records water usage and filling history using Python data structures like lists and tuples. This stored information can be used for analysis to understand consumption patterns.

The project also demonstrates the effective use of Python programming concepts such as variables, conditional statements, loops, functions, and data structures. These concepts work together to build the logic of the system. The modular structure of the program makes it easy to understand, modify, and extend in the future. Each part of the system performs a specific task, which improves clarity and maintainability.

6.2 FUTURE SCOPE

The Water Tank Level Monitoring and Usage System has a wide scope for future enhancements and real-world applications. Currently, the system works as a Python-based simulation using basic logic and data structures, but it can be upgraded into a more advanced and practical system using modern technologies. One of the major future improvements is the integration of IoT (Internet of Things). By using real sensors such as ultrasonic or water level sensors, the system can monitor actual water levels in real time instead of simulated values. This will make the system more accurate and useful for real-world applications in homes, agriculture, and industries.

Another enhancement is the use of a database system like MySQL or SQLite. At present, data is stored temporarily using lists and tuples, which gets lost after program execution. A database will allow permanent storage of water usage and tank filling history, enabling better analysis and long-term record keeping.

The system can also be improved by adding a mobile or web application interface. This will help users monitor and control the water tank remotely and receive alerts when the water level is low or full.

REFERENCE

- Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace Independent Publishing Platform.
  ISBN: 978-1441412690
- Matthes, E. (2023). Python Crash Course: A Hands-On, Project-Based Introduction to Programming (3rd Edition). No Starch Press.
  ISBN: 978-1718502703
- Zelle, J. M. (2017). Python Programming: An Introduction to Computer Science (3rd Edition). Franklin, Beedle & Associates.
  ISBN: 978-1590282755
- Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2nd Edition). Green Tea Press.
  ISBN: 978-1491939369

APPENDICES

Appendix A-Source Code :

```
import random
import time
from datetime import datetime

# ---------------------------------------------
# SETTINGS
# ---------------------------------------------
LOW = 30
HIGH = 90

# ---------------------------------------------
# SENSOR (SIMULATED)
# ---------------------------------------------
def read_sensor():
    return random.randint(20, 80)

# ---------------------------------------------
# INITIAL STATE
# ---------------------------------------------
water_level = read_sensor()
motor_on = False
total_usage = 0
usage_history = []
fill_history = []

# ---------------------------------------------
# HEADER WITH DATE & TIME
# ---------------------------------------------
def header():
    now = datetime.now()
    print("\n" + "=" * 50)
    print("💧 SMART WATER TANK SYSTEM 💧")
    print("📅 Date:", now.strftime("%d-%m-%Y"))
    print("🕒 Time:", now.strftime("%H:%M:%S"))
    print("=" * 50)

# ---------------------------------------------
# MODULE 1: MONITORING
# ---------------------------------------------
```

```
def monitor():

    print("\n📡 SENSOR STATUS")
    print(f"Water Level : {water_level}%")
    return water_level

# ------------------------------------------
# MODULE 2: MOTOR CONTROL
# ------------------------------------------
def control(level):
    global motor_on

    print("\n⚙ MOTOR CONTROL") if level

    <= LOW:
        motor_on = True
        print("⚠ Low Level → Motor ON")

    elif level >= HIGH:
        motor_on = False
        print("✔ Tank Full → Motor OFF")

    else:
        print("✓ Level Normal")

# ------------------------------------------
# MODULE 3: USER ACTION + VALIDATION + DATA
# ------------------------------------------
def user_action():
    global water_level, total_usage, usage_history, fill_history

    print("\n👤 USER MENU")
    print("1. Use Water")
    print("2. Fill Tank")
    print("3. View Usage History")
    print("4. View Fill History")
    print("5. Exit")

    choice = input("Enter choice: ")

    # -------- USE WATER --------
    if choice == "1":
        used = int(input("Enter water used (%): "))
```

```
        if used > water_level:

            print("❌ ERROR: Not enough water in tank!")
            print(f"Available water: {water_level}%")
            print("🚨 ALERT: Usage exceeds available water")
        else:
            water_level -= used
            total_usage += used

            now = datetime.now().strftime("%H:%M:%S")
            usage_history.append((used, now))

            print(f"🚿 Used {used}% water at {now}")

    # -------- FILL WATER --------
    elif choice == "2":
        if motor_on:
            added = int(input("Enter water to add (%): "))

            if water_level + added > 100:
                print("❌ ERROR: Tank will overflow!")
                print(f"Current level: {water_level}%")
                print(f"You can only add up to {100 - water_level}%")
                print("🚨 ALERT: Overflow prevented")
            else:
                water_level += added

                now = datetime.now().strftime("%H:%M:%S")
                fill_history.append((added, now))

                print(f"💧 Added {added}% water at {now}")
        else:
            print("❌ Motor OFF → Cannot Fill")

    # -------- VIEW USAGE HISTORY --------
    elif choice == "3":
        print("\n📊 USAGE HISTORY")
        if not usage_history:
            print("No usage yet")
        else:
            for u, t in usage_history:
                print(f"Used {u}% at {t}")

    # -------- VIEW FILL HISTORY --------
```

```
    elif choice == "4":

        print("\n💧 FILL HISTORY")
        if not fill_history:
            print("No filling yet")
        else:
            for f, t in fill_history:
                print(f"Added {f}% at {t}")

    # --------- EXIT ---------
    elif choice == "5":
        return False

    else:
        print("❌ Invalid Choice")

    water_level = max(0, min(100, water_level))
    return True


# ---------------------------------------------
# MODULE 4: DISPLAY
# ---------------------------------------------
def display():
    print("\n📊 SYSTEM STATUS")
    print("-" * 40)
    print(f"Water Level : {water_level}%")
    print(f"Motor       : {'ON' if motor_on else 'OFF'}")
    print(f"Total Usage : {total_usage} units")
    print("-" * 40)


# ---------------------------------------------
# MODULE 5: ALERT SYSTEM (DETAILED)
# ---------------------------------------------
def alert():
    print("\n🚨 ALERT SYSTEM")

    if water_level <= 5:
        print("🔴 CRITICAL: Tank Almost Empty (0-5%)")

    elif water_level <= 15:
        print("🟥 VERY LOW: Water Level Very Low (6-15%)")
```

```
    elif water_level <= 30:

        print("☐ LOW: Water Level Low (16-30%)")

    elif water_level <= 60:
        print("☐ NORMAL: Water Level Medium (31-60%)")

    elif water_level <= 85:
        print("🔘 HIGH: Water Level High (61-85%)")

    elif water_level < 100:
        print("☐ VERY HIGH: Tank Almost Full (86-99%)")

    elif water_level == 100:
        print("🔘 FULL: Tank Completely Full (100%)")

# ------------------------------------------
# MAIN SYSTEM
# ------------------------------------------

header()
print("System Started...")

# FIRST SENSOR DISPLAY
level = monitor()
control(level)
display()
alert()

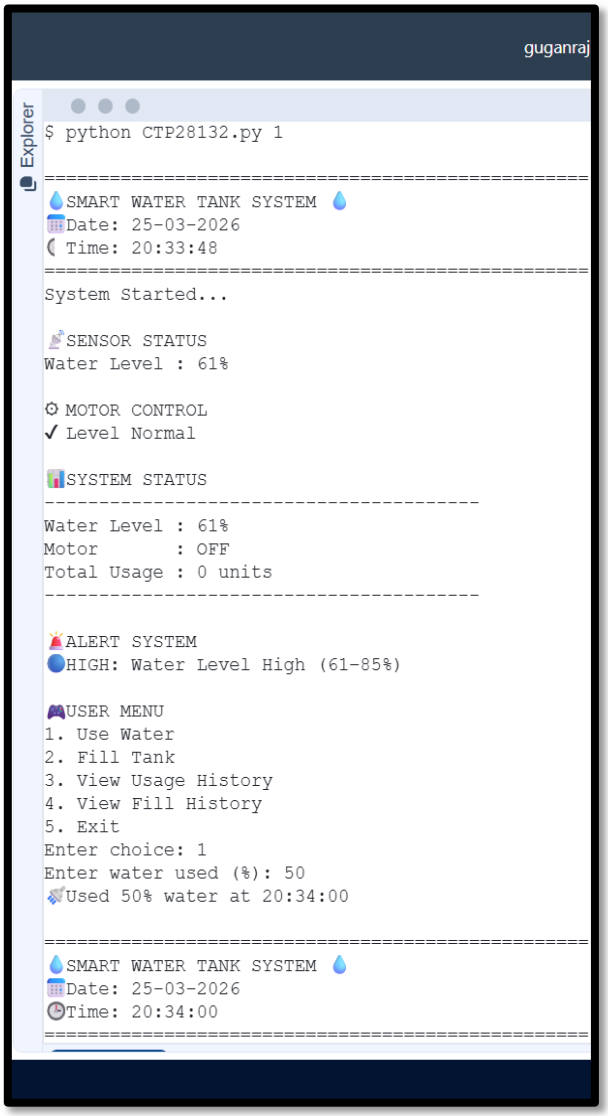
# INTERACTIVE LOOP
while True:
    if not user_action():
        print("\n✋ System Stopped")
        break

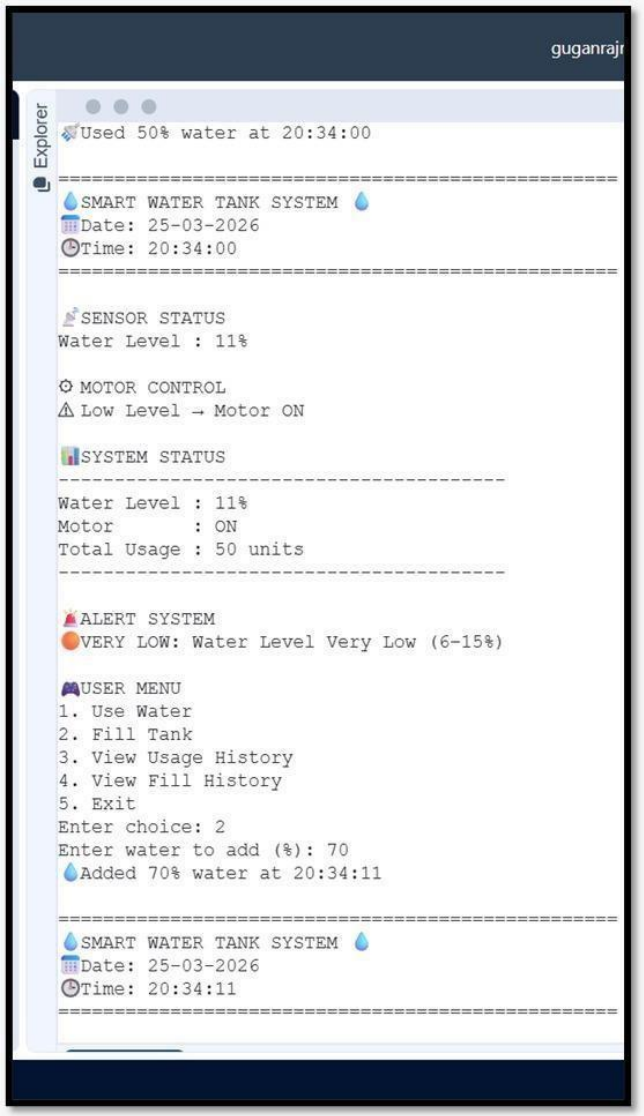
    header()
    level = monitor()
    control(level)
    display()
    alert()

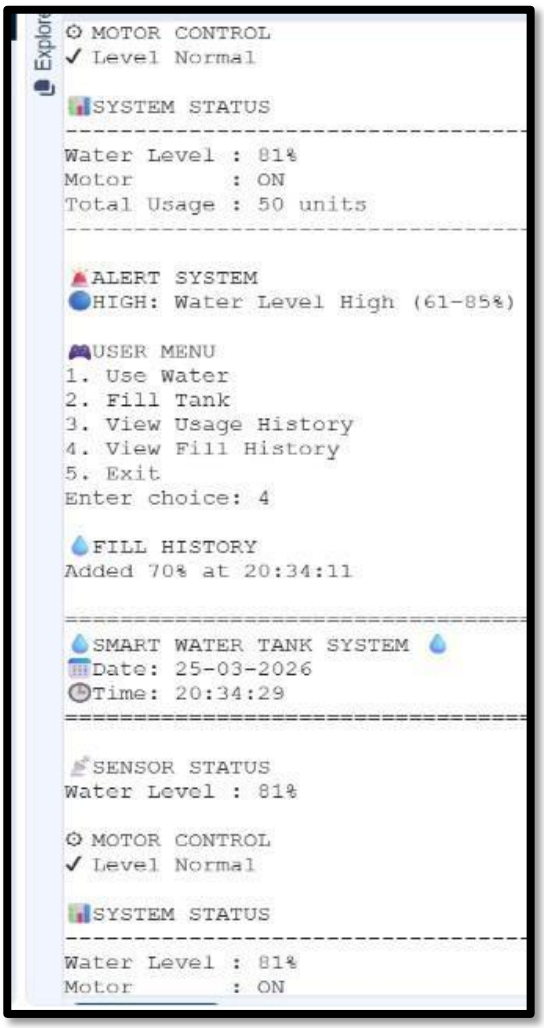
    time.sleep(1)
```

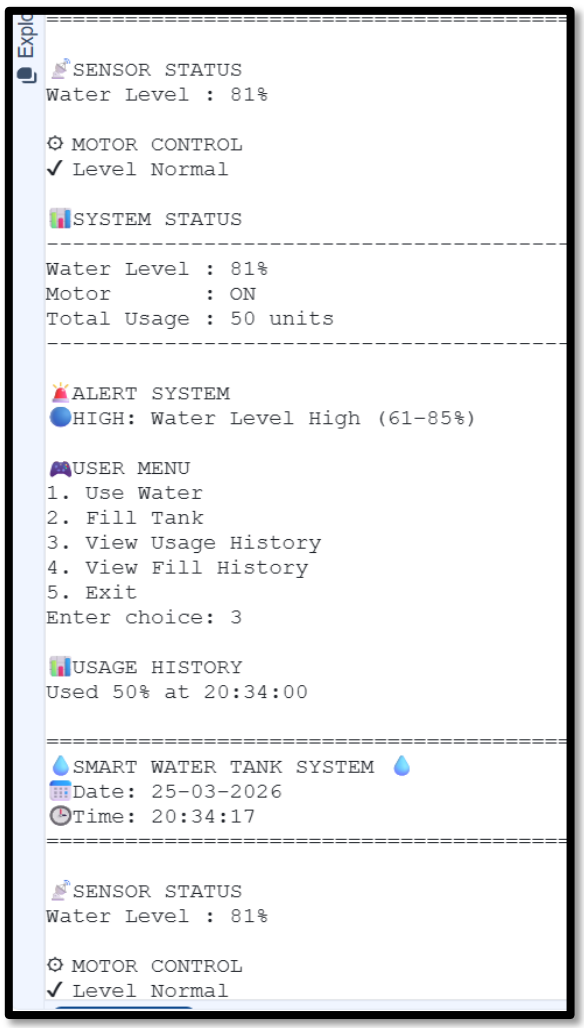
APPENDICES

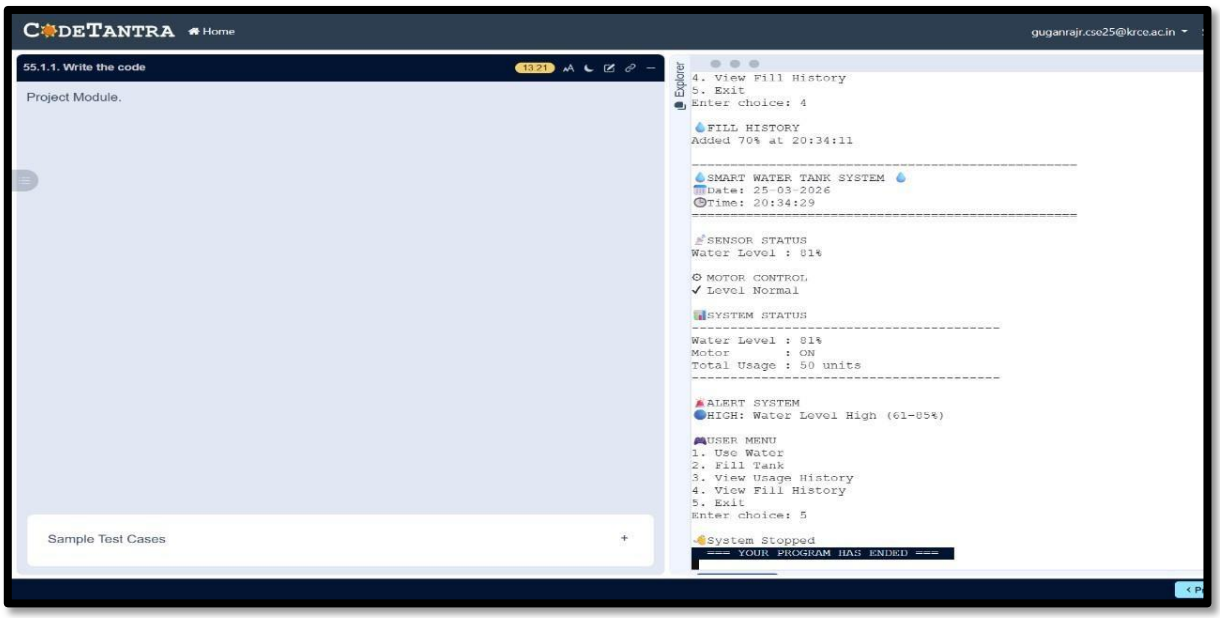
Appendix B-Screenshots :






⚙ MOTOR CONTROL
✓ Level Normal
SYSTEM STATUS
Water Level : 81%
Motor : ON
Total Usage : 50 units
ALERT SYSTEM
HIGH: Water Level High (61-85%)
USER MENU
1. Use Water
2. Fill Tank
3. View Usage History
4. View Fill History
5. Exit
Enter choice: 4
FILL HISTORY
Added 70% at 20:34:11
SMART WATER TANK SYSTEM
Date: 25-03-2026
Time: 20:34:29
SENSOR STATUS
Water Level : 81%
⚙ MOTOR CONTROL
✓ Level Normal
SYSTEM STATUS
Water Level : 81%
Motor : ON


SENSOR STATUS
Water Level : 81%
⚙ MOTOR CONTROL
✓ Level Normal
SYSTEM STATUS
Water Level : 81%
Motor : ON
Total Usage : 50 units
ALERT SYSTEM
HIGH: Water Level High (61-85%)
USER MENU
1. Use Water
2. Fill Tank
3. View Usage History
4. View Fill History
5. Exit
Enter choice: 3
USAGE HISTORY
Used 50% at 20:34:00
SMART WATER TANK SYSTEM
Date: 25-03-2026
Time: 20:34:17
SENSOR STATUS
Water Level : 81%
⚙ MOTOR CONTROL
✓ Level Normal


CODETANTRA
Home
guganrajr.cse25@krce.ac.in
55.1.1. Write the code
Project Module.
Sample Test Cases
4. View Fill History
5. Exit
Enter choice: 4
FILL HISTORY
Added 70% at 20:34:11
SMART WATER TANK SYSTEM
Date: 25-03-2026
Time: 20:34:29
SENSOR STATUS
Water Level : 81%
MOTOR CONTROL
Level Normal
SYSTEM STATUS
Water Level : 81%
Motor : ON
Total Usage : 50 units
ALERT SYSTEM
HIGH: Water Level High (61-85%)
USER MENU
1. Use Water
2. Fill Tank
3. View Usage History
4. View Fill History
5. Exit
Enter choice: 5
System Stopped
=== YOUR PROGRAM HAS ENDED ===